

---

aliases: - recursion - 递归函数

---

# 递归函数：函数自己调用自己

---

前置：已掌握函数定义、参数、返回值、if 判断 目标：能写出带终止条件的递归函数，理解实验题

`f(n)` 求 `1~n` 之和 用时：约 10 分钟

相关笔记：[函数基础](#) · [条件判断](#) · [循环深入](#)

---

## 一、递归是什么

递归就是：一个函数在函数体内部调用自己。

```
def count_down(n):  
    if n == 0:  
        print("结束")  
        return  
  
    print(n)  
    count_down(n - 1)  
  
count_down(3)
```

执行过程：

```
count_down(3)  
    print(3)  
count_down(2)  
    print(2)  
count_down(1)  
    print(1)
```

```
count_down(0)

print("结束")
```

记忆句：递归 = 把大问题拆成一个小一点的同类问题。

---

## 二、递归必须有两个部分

### 1. 终止条件

告诉函数：什么时候别再调用自己了。

```
if n == 1:
    return 1
```

没有终止条件，函数会一直调用自己，最后报错：

```
RecursionError: maximum recursion depth exceeded
```

### 2. 递归关系

告诉函数：当前问题如何变成更小的问题。

```
return n + f(n - 1)
```

这句话的意思是：

$1 \sim n$  的和 =  $n + 1 \sim (n-1)$  的和

---

## 三、实验题：递归求 $1 \sim n$ 之和

题目：编写函数 `f(n)`，输入 `n`，求  $1 \sim n$  之和，要求使用递归。

```
def f(n):  
    if n == 1:  
        return 1  
  
    return n + f(n - 1)  
  
num = int(input("请输入 n: "))  
print(f"1~{num} 的和是: {f(num)}")
```

以 `f(5)` 为例：

```
f(5)  
= 5 + f(4)  
= 5 + 4 + f(3)  
= 5 + 4 + 3 + f(2)  
= 5 + 4 + 3 + 2 + f(1)  
= 5 + 4 + 3 + 2 + 1  
= 15
```

**关键点：**每次调用都让 `n` 变小，直到 `n == 1`。

---

## 四、递归和循环的对比

递归写法：

```
def f(n):  
    if n == 1:  
        return 1  
  
    return n + f(n - 1)
```

循环写法：

```
def f_loop(n):  
    total = 0
```

```
for i in range(1, n + 1):  
    total += i  
  
return total
```

对比：

| 写法 | 优点                | 缺点                |
|----|-------------------|-------------------|
| 递归 | 思路像数学公式，适合树、分治、阶乘 | 层数太深会报错，初学容易忘终止条件 |
| 循环 | 更直观、更省内存          | 有些复杂结构写起来不如递归自然   |

考试里如果要求“使用递归”，就不能用 `for` 或 `while` 直接累加。

## 五、经典例子：阶乘

阶乘公式：

$$n! = n * (n-1) * (n-2) * \dots * 1$$

递归关系：

$$n! = n * (n-1)!$$

代码：

```
def factorial(n):  
    if n == 1:  
        return 1  
  
    return n * factorial(n - 1)  
  
print(factorial(5)) # 120
```

展开过程：

```
factorial(5)
= 5 * factorial(4)
= 5 * 4 * factorial(3)
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1
= 120
```

## 六、递归常见错误

### 错误 1：没有终止条件

```
def f(n):
    return n + f(n - 1)
```

问题：函数永远不知道什么时候停。

### 错误 2：参数没有变小

```
def f(n):
    if n == 1:
        return 1
    return n + f(n)
```

问题：每次还是 `f(n)`，没有靠近终止条件。

### 错误 3：终止条件写错

```
def f(n):
    if n == 0:
        return 0
    return n + f(n - 1)
```

这段代码本身可以算  $1^n$ ，但如果题目要求  $n$  是正整数，通常更自然写成：

```
if n == 1:  
    return 1
```

如果要兼容  $n == 0$ ，可以写：

```
def f(n):  
    if n <= 0:  
        return 0  
    return n + f(n - 1)
```

## 七、怎么判断一道题能不能用递归

看它能不能拆成：

当前问题 = 当前一步 + 更小的同类问题

适合递归的题：

- $1^n$  求和：  $f(n) = n + f(n-1)$
- 阶乘：  $\text{factorial}(n) = n * \text{factorial}(n-1)$
- 倒计时：  $\text{count\_down}(n)$  调  $\text{count\_down}(n-1)$
- 树结构、目录遍历、分治算法

不太适合初学者用递归硬写的题：

- 简单列表遍历
- 普通菜单循环
- 学生管理系统这种交互式程序

## 八、考试速记

递归函数三步：

1. 写函数名和参数
2. 写终止条件
3. 写递归调用，让参数向终止条件靠近

## 模板：

```
def 函数名(n):  
    if 终止条件:  
        return 最简单情况的答案  
  
    return 当前一步 + 函数名(更小的问题)
```

## 实验 7 的递归求和模板：

```
def f(n):  
    if n == 1:  
        return 1  
    return n + f(n - 1)
```

## 一句话复述：

递归就是函数自己调用自己，但必须有终止条件，并且每次调用都要让问题变小。